Tutorials

API Health Check Endpoint: How to Build One (Next.js, Express, FastAPI)

Fabián Delgado · March 30, 2026 · 11 min read

An API health check endpoint tells you if your service is alive before your users tell you it's dead. Build one that checks DB, Redis, memory, and external services, with full code examples.



Your API returns `200 OK` on every request. The database is connected. Redis is running. Everything works.

Nurbak is an AI pentester on its own model: connect GitHub and it finds the vulnerabilities in your code, with file, line and the fix as a Pull Request.

→ Scan your repo for free

Then your PostgreSQL connection pool fills up. Requests start queuing. Response times climb from 50ms to 8 seconds. Your app is technically "up" but practically unusable. The load balancer keeps sending traffic because it doesn't know anything is wrong.

A health check endpoint would have caught this in seconds. Here's how to build one that actually works, with full code examples for Next.js, Express, and FastAPI.

What a Health Check Endpoint Actually Is

A health check is a dedicated route, usually `/health` or `/api/health` that reports whether your application can do its job. Not just "is the process running," but "can this server handle real requests right now."

Three things consume health check endpoints:

- **Load balancers** (AWS ALB, Nginx, Cloudflare), route traffic away from unhealthy instances
- **Container orchestrators** (Kubernetes, ECS, Docker), restart failing containers
- **Monitoring tools** (UptimeRobot, Datadog), alert you when something breaks

If you only return `{ "status": "ok" }`, you're testing that the process is alive. That's the easy part. The hard part (the part that actually prevents outages) is checking dependencies.

What a Good Health Check Should Include

A production health check verifies everything your API needs to serve real requests:

CHECK	WHAT IT CATCHES	HOW TO TEST IT
Database	Connection pool exhaustion, network issues, replication lag	Run `SELECT 1` with a timeout
Cache (Redis/Memcached)	Eviction storms, connection failures, high memory	Run `PING` with a timeout
External APIs	Third-party outages (Stripe, Auth0, S3)	HEAD request or lightweight endpoint call
Memory	Memory leaks, OOM risk	Check `process.memoryUsage()` against a threshold
Disk (if applicable)	Full disk, log rotation failures	Check available space

Each check should have a **timeout**. A health check that hangs for 30 seconds waiting for a dead database is worse than no health check at all. Keep the total response time under 3 seconds.

Implementation: Next.js App Router

Create `app/api/health/route.ts`. For a deeper Next.js-specific walkthrough with App Router, Pages Router, and multi-region monitoring, see our complete Next.js health check guide.

```
// app/api/health/route.ts
import { NextResponse } from 'next/server'

interface HealthCheck {
  name: string
  status: 'healthy' | 'unhealthy'
  latency: number
```

```
  message?: string
}

async function checkWithTimeout(
  name: string,
  fn: () => Promise<void>,
  timeoutMs = 3000
): Promise<HealthCheck> {
  const start = Date.now()
  try {
await Promise.race([
  fn(),
  new Promise((_, reject) =>
    setTimeout(() => reject(new Error('Timeout')), timeoutMs)
  ),
])
return {
  name,
  status: 'healthy',
  latency: Date.now() - start,
}
  } catch (error) {
return {
  name,
  status: 'unhealthy',
  latency: Date.now() - start,
  message: error instanceof Error ? error.message : 'Unknown error',
}
  }
}

export async function GET() {
  const startTime = Date.now()

  const checks = await Promise.all([
// Database check
checkWithTimeout('database', async () => {
  const { Pool } = await import('pg')
  const pool = new Pool({
    connectionString: process.env.DATABASE_URL,
  })
  const client = await pool.connect()
  try {
```

```
    await client.query('SELECT 1')
  } finally {
    client.release()
    await pool.end()
  }
}),

// Redis check
checkWithTimeout('redis', async () => {
  const { createClient } = await import('redis')
  const client = createClient({ url: process.env.REDIS_URL })
  await client.connect()
  try {
    await client.ping()
  } finally {
    await client.quit()
  }
}),

// Memory check
checkWithTimeout('memory', async () => {
  const usage = process.memoryUsage()
  const heapUsedMB = usage.heapUsed / 1024 / 1024
  const heapTotalMB = usage.heapTotal / 1024 / 1024
  const usagePercent = (heapUsedMB / heapTotalMB) * 100

  if (usagePercent > 90) {
    throw new Error(
      `Heap usage at ${usagePercent.toFixed(1)}% (${heapUsedMB.toFixed(0)}MB)`
    )
  }
}),
  ])

  const isHealthy = checks.every((c) => c.status === 'healthy')

  const response = {
status: isHealthy ? 'healthy' : 'unhealthy',
timestamp: new Date().toISOString(),
totalLatency: Date.now() - startTime,
uptime: process.uptime(),
checks,
  }
```

```
  return NextResponse.json(response, {
status: isHealthy ? 200 : 503,
headers: {
  'Cache-Control': 'no-cache, no-store, must-revalidate',
},
  })
}
```

Key details:

- `Cache-Control: no-cache` load balancers and CDNs must never cache health checks
- `503 Service Unavailable` when unhealthy, this is the standard status code that load balancers expect
- Each check runs in parallel with `Promise.all` the total time is the slowest check, not the sum
- 3-second timeout per check prevents the health endpoint from hanging

Example response when healthy:

```
{
  "status": "healthy",
  "timestamp": "2026-03-30T14:22:01.234Z",
  "totalLatency": 45,
  "uptime": 86400,
  "checks": [
{ "name": "database", "status": "healthy", "latency": 12 },
{ "name": "redis", "status": "healthy", "latency": 3 },
{ "name": "memory", "status": "healthy", "latency": 0 }
  ]
}
```

Example response when the database is down:

```
{
  "status": "unhealthy",
```

```
  "timestamp": "2026-03-30T14:22:01.234Z",
  "totalLatency": 3004,
  "uptime": 86400,
  "checks": [
{ "name": "database", "status": "unhealthy", "latency": 3001, "message": "Timeou
{ "name": "redis", "status": "healthy", "latency": 3 },
{ "name": "memory", "status": "healthy", "latency": 0 }
  ]
}
```

Scan your repo in minutes

Your code never goes to OpenAI or Anthropic. No credit card.

Connect GitHub →

Implementation: Express

Same logic, adapted for Express:

```
// routes/health.js
const express = require('express')
const router = express.Router()

async function checkWithTimeout(name, fn, timeoutMs = 3000) {
  const start = Date.now()
  try {
await Promise.race([
  fn(),
  new Promise((_, reject) =>
    setTimeout(() => reject(new Error('Timeout')), timeoutMs)
  ),
])
return { name, status: 'healthy', latency: Date.now() - start }
  } catch (error) {
return {
  name,
```

```
  status: 'unhealthy',
  latency: Date.now() - start,
  message: error.message,
}
  }
}

router.get('/health', async (req, res) => {
  const startTime = Date.now()

  const checks = await Promise.all([
// Database (using your existing pool)
checkWithTimeout('database', async () => {
  const pool = req.app.get('dbPool') // or import your pool
  await pool.query('SELECT 1')
}),

// Redis
checkWithTimeout('redis', async () => {
  const redis = req.app.get('redisClient')
  await redis.ping()
}),

// Memory
checkWithTimeout('memory', async () => {
  const usage = process.memoryUsage()
  const pct = (usage.heapUsed / usage.heapTotal) * 100
  if (pct > 90) throw new Error(`Heap at ${pct.toFixed(1)}%`)
}),
  ])

  const isHealthy = checks.every((c) => c.status === 'healthy')

  res
.status(isHealthy ? 200 : 503)
.set('Cache-Control', 'no-cache, no-store, must-revalidate')
.json({
  status: isHealthy ? 'healthy' : 'unhealthy',
  timestamp: new Date().toISOString(),
  totalLatency: Date.now() - startTime,
  uptime: process.uptime(),
  checks,
})
```

```
})

module.exports = router
```

Mount it in your app:

```
const healthRouter = require('./routes/health')
app.use('/api', healthRouter)
// Health check available at GET /api/health
```

Implementation: FastAPI

Python version with async support:

```
# routes/health.py
import asyncio
import time
import psutil
from datetime import datetime, timezone
from fastapi import APIRouter
from fastapi.responses import JSONResponse

router = APIRouter()


async def check_with_timeout(name: str, coro, timeout_s: float = 3.0):
start = time.monotonic()
try:
    await asyncio.wait_for(coro(), timeout=timeout_s)
    return {
        "name": name,
        "status": "healthy",
        "latency": round((time.monotonic() - start) * 1000),
    }
except Exception as e:
    return {
        "name": name,
        "status": "unhealthy",
        "latency": round((time.monotonic() - start) * 1000),
```

```
        "message": str(e),
    }


async def check_database():
"""Checks PostgreSQL with asyncpg."""
import asyncpg
import os

conn = await asyncpg.connect(os.environ["DATABASE_URL"])
try:
    await conn.fetchval("SELECT 1")
finally:
    await conn.close()


async def check_redis():
"""Checks Redis with redis-py async."""
import redis.asyncio as aioredis
import os

client = aioredis.from_url(os.environ.get("REDIS_URL", "redis://localhost"))
try:
    await client.ping()
finally:
    await client.aclose()


async def check_memory():
"""Checks memory usage via psutil."""
memory = psutil.virtual_memory()
if memory.percent > 90:
    raise Exception(f"Memory at {memory.percent}%")


@router.get("/health")
async def health_check():
start_time = time.monotonic()

checks = await asyncio.gather(
    check_with_timeout("database", check_database),
    check_with_timeout("redis", check_redis),
    check_with_timeout("memory", check_memory),
```

```
    )

    is_healthy = all(c["status"] == "healthy" for c in checks)

    body = {
        "status": "healthy" if is_healthy else "unhealthy",
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "total_latency": round((time.monotonic() - start_time) * 1000),
        "checks": checks,
    }

    return JSONResponse(
        content=body,
        status_code=200 if is_healthy else 503,
        headers={"Cache-Control": "no-cache, no-store, must-revalidate"},
    )
```

Register it in your app:

```
from fastapi import FastAPI
from routes.health import router as health_router

app = FastAPI()
app.include_router(health_router, prefix="/api")
```

Common Mistakes to Avoid

After building hundreds of health checks, these are the patterns that cause the most production incidents:

1. No timeout on dependency checks

If your database is slow (not down, just slow), a health check without a timeout will hang for 30+ seconds. The load balancer times out, retries, and eventually marks *every* instance as unhealthy. Use a 3-second timeout on each individual check.

2. Caching the health response

Never cache health check responses. A CDN caching `200 OK` for 60 seconds means a full minute of traffic routed to a dead instance. Always set `Cache-Control: no-cache, no-store, must-revalidate`.

3. Checking too much

Your health check should verify dependencies that are critical to serving requests. Don't check optional services (analytics, feature flags) or external APIs that have their own redundancy. A failed Mixpanel connection shouldn't make your app "unhealthy."

4. Exposing sensitive information

Don't include database connection strings, internal IPs, or stack traces in health check responses. Return the error *message*, not the error *details*. Consider a lightweight public endpoint (`/health`) and a detailed private one (`/health/detailed`) behind authentication.

5. Health check as the only monitoring

A health check tells you if the server *can* handle requests. It doesn't tell you if real requests are *actually* succeeding. Your `/api/health` can return 200 while `/api/checkout` returns 500, because the health check doesn't test your business logic.

This is where you need request-level monitoring on top of health checks.

Beyond Health Checks: Monitoring Every API Route

A health check runs every 30-60 seconds and tests a synthetic request. That's a good baseline. But it misses:

- **Intermittent errors**if 3% of requests to `/api/checkout` fail, a health check will likely never hit that 3%

- **Latency spikes**your health check measures empty response time, not the P95 of real requests with actual data
- **Endpoint-specific failures** `/api/users` works fine, `/api/payments` is broken. The health check tests neither.

The fix is to measure *every* API route from real traffic, not only synthetic pings. In Next.js you can register OpenTelemetry in the instrumentation hook and send the data to the metrics backend or APM you already use:

```
// instrumentation.ts
import { registerOTel } from '@vercel/otel'

export function register() {
  registerOTel({ serviceName: 'my-app' })
}
```

With request-level data in place, you can track:

- Every API route (including your health check endpoint)
- P50/P95/P99 latency per endpoint from real traffic
- Error rates and status code distribution
- Alerts per route when error rate or latency crosses a threshold

Think of it this way: the health check is your smoke detector. Request-level monitoring is the sensor in every room.

Quick Reference: Health Check by Framework

FRAMEWORK	FILE	ROUTE	DB LIBRARY
Next.js App Router	`app/api/health/route.ts`	`GET /api/health`	`pg`
Express	`routes/health.js`	`GET /api/health`	`pg`

FRAMEWORK	FILE	ROUTE	DB LIBRARY
FastAPI	`routes/health.py`	`GET /api/health`	`asyncpg`

All three implementations share the same pattern: parallel checks with timeouts, JSON response, 200 or 503, no caching.

Get Started

Build the health check endpoint, pick the framework tab above and copy the code. Then add real monitoring on top:

1. Point an external uptime monitor at `/api/health` with a 1-minute interval
2. Instrument your API routes (for example with OpenTelemetry) to get latency and error rates from real traffic
3. Set alerts per route, sent to the channel your on-call engineer actually watches
4. Deploy

Your health check tells you the server is alive. Request-level monitoring tells you if it's actually working.

Related Articles

- REST API monitoring guide
- API gateway monitoring

F

Fabián Delgado

The Nurbak Team builds developer-first API monitoring tools. We share insights on uptime, performance, alerting, and best practices for keeping APIs healthy in production.

Ready to try it on your code?

Nurbak finds exploitable vulnerabilities in your repo with its own AI model and opens the Pull Request with the fix and a security test.

Create account and connect GitHub

Read Next

Tutorials

Next.js instrumentation.ts: A Complete Guide to Server-Side Monitoring

April 7, 2026 · 11 min read

Tutorials

API Testing Strategies: A Practical Guide for Developers in 2026

April 3, 2026 · 12 min read

Tutorials

API Security Testing: OWASP Top 10, Tools & Best Practices

April 3, 2026 · 13 min read

About the author

Fabián Delgado, Founder & CTO of Nurbak

Engineering Manager with +15 years building critical systems at Rappi, Tiendamia, Ueno Bank and AGESIC (Uruguay's e-government agency). Independent AI consultant and builder of open-source MCP servers for government data.

fabiandelgado.uy · LinkedIn · GitHub · Wikidata

← All posts

© 2026 Nurbak. All rights reserved.